Experiment 5

Student Name: Jayanaath S
UID: 23BCC70022
Branch: AIT-CSE
Section: 23AIT_KRG-1 A
Semester: 6
Date of Performance: 13/02/2026
Subject Name: Full Stack II
Subject Code: 23CSH-382

- **Aim:**
  The aim of this experiment is to verify the correctness, stability, and reliability of the **EcoTrack** React application by implementing automated testing techniques including unit testing, snapshot testing, component rendering tests, Redux reducer testing, and mocking using Jest, React Testing Library, and Redux Mock Store.

- **Objectives:**
  The main objectives of this experiment are as follows:
  1. To understand the importance of automated testing in modern frontend applications.
  2. To write unit tests for JavaScript logic using Jest.
  3. To validate UI rendering using React Testing Library.
  4. To perform snapshot testing to detect unintended UI structure changes.
  5. To test Redux reducers independently for correct state transitions.
  6. To mock Redux store behavior using redux-mock-store.
  7. To simulate application state for component-level testing.
  8. To verify loading and state transition logic in asynchronous Redux flows.
  9. To analyze test failures systematically and debug errors effectively.
  10. To improve software maintainability and confidence through test coverage.

- **Implementation:**

The testing phase of the EcoTrack application was implemented through the following structured steps:

1. Component Rendering Test (Button Component)
   - The custom Button component was tested to verify whether it renders correctly with given props.
   - The component was rendered using render() from React Testing Library.
   - The DOM was queried using screen.getByText().
   - The presence of the button text was validated using Jest-DOM matchers.

2. Snapshot Testing
   - Snapshot testing was implemented to capture the UI structure of the Button component and detect unintended structural changes in future updates.
   - The rendered container was captured.
   - toMatchSnapshot() was used to generate and compare snapshot files.

3. Redux Mock Store Testing (Logs Component)
   - To test the Logs component without relying on the real store, a mock Redux store was created using redux-mock-store.
   - Thunk middleware was included to allow dispatching async actions.
   - A predefined state was injected into the store.
   - The component was wrapped inside <Provider>.
   - This ensured the UI correctly rendered based on mocked Redux state.

4. <u>Reducer Unit Testing</u>

   The Redux reducer (logSlice) was tested independently to validate:

   - Initial state correctness
   - Proper state transition during async pending state
   - These tests confirmed correct reducer behavior during asynchronous lifecycle stages.

- **Output:**

logSlice.test.js × logSlice.jsx

EcoTrack > src > store > logSlice.test.js > test("testing loading state when fetch is pe

```
import logReducer from "./logSlice";

test("testing initial state of store",()=>{
  const initialState={
      data:[],
      status:"idle",
      error:null,
    };
  const result=logReducer(undefined,{type:undefined});
  expect(result).toEqual(initialState);
})

test("testing loading state when fetch is pending",()=>{
  const initialState={
    data:[],
    status:"idle",
    error:null,
  };
  const action={type:"logs/fetchlogs/pending"};
  const result=logReducer(initialState,action);
  console.log(result);
  expect(result.status).toBe("loading");
})
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
 ✓ testing initial state of store 2ms
 ✓ testing loading state when fetch is pending 1ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  19:25:18
   Duration  99ms

 PASS  Waiting for file changes...
       press h to show help, press q to quit
```

logSlice.test.jsx Button.test.jsx × Button.jsx

EcoTrack > src > components > Button.test.jsx > ...

```
import {screen,render} from "@testing-library/react"
import Button from "./Button"
import "@testing-library/jest-dom";

test("testing if button rendered properly",()=>{
  render(<Button textVal="Fetch logs"/>);
  expect(screen.getByText(/Fetch logs/)).toBeInTheDocument();
})

test("testing button UI snapshot",()=>{
  const {container}=render(<Button textVal="Fetch logs"/>);
  expect(container).toMatchSnapshot();
})
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
 ✓ src/components/Button.test.jsx (2 tests) 43ms
   ✓ testing if button rendered properly 37ms
   ✓ testing button UI snapshot 4ms

  Snapshots  1 written
 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  19:42:45
   Duration  215ms
```

logSlice.test.jsx | Button.test.jsx | Logs.test.jsx | store.jsx

EcoTrack > src > pages > Logs.test.jsx > ...

```
import { render, screen } from "@testing-library/react";
import { Provider } from "react-redux";
import configureStore from "redux-mock-store";
import {thunk} from "redux-thunk";
import Logs from "./Logs";
import "@testing-library/jest-dom";

// create mock store with thunk middleware
const mockStore = configureStore([thunk]);

test("mocking fetchLog api", () => {

  // manually provide Redux state
  const store = mockStore({
    logs: {
      data: [
        { id: 1, activity: "Plane Travel", carbon: 4 },
        { id: 2, activity: "Electricity Usage", carbon: 6 },
        { id: 3, activity: "Cycling", carbon: 0 },
      ],
      status: "success",
      error: null,
    },
  });

```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
✓ src/pages/Logs.test.jsx (1 test) 38ms
  ✓ mocking fetchLog api 37ms

Test Files  1 passed (1)
     Tests  1 passed (1)
  Start at  20:33:03
  Duration  232ms

PASS  Waiting for file changes...
      press h to show help, press q to quit
```

- **Results:**

The EcoTrack application was successfully validated using automated testing techniques. Component-level tests ensured correct UI rendering, snapshot testing protected against unintended structural modifications, and Redux reducer tests verified accurate state transitions. Mock store testing confirmed that components behave correctly under simulated application states.

This experiment strengthened application reliability, reduced dependency on manual testing, and improved overall code confidence and maintainability.

- **Learning Outcomes:**
  After completing this experiment, I have learnt to:
  1. Write unit tests for React components using React Testing Library.
  2. Perform snapshot testing to track UI changes.
  3. Use Jest matchers to validate expected outputs.
  4. Test Redux reducers independently for state correctness.
  5. Simulate Redux store behaviour using redux-mock-store.
  6. Understand the role of middleware like Redux Thunk in async flows.
  7. Debug failing test cases systematically.
  8. Separate concerns while testing UI, reducers, and async logic.
  9. Improve application stability through automated validation.
  10. Adopt industry-level frontend testing practices.